GEN-AI Handout - 1

Transformers

The Transformer is a type of neural network architecture designed to handle sequential data, such as text, but unlike traditional models like RNNs or LSTMs, it does not process data in order. Instead, it relies entirely on a mechanism called self-attention to draw global dependencies between input and output.

Input Representation:

Tokenization:

The input sentence (or sequence) is broken into smaller units called tokens. These tokens can be words or sub words, depending on the tokenization scheme (e.g., Word Piece, Byte-Pair Encoding).

Embedding:

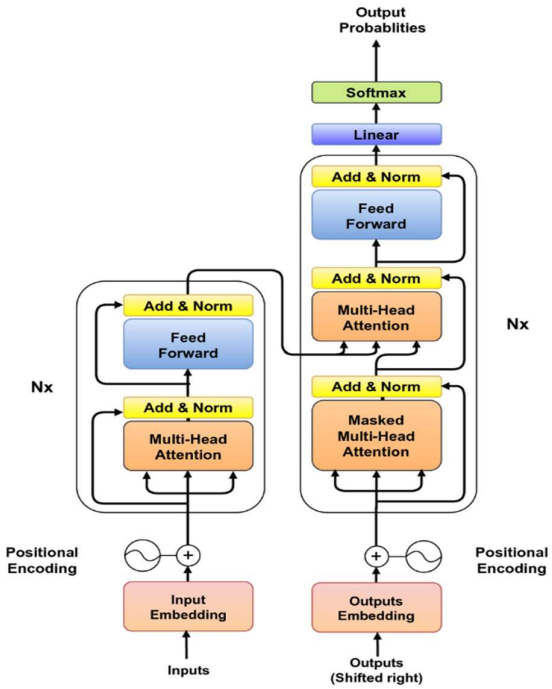
Each token is mapped to a high-dimensional vector (called word embeddings). These embeddings capture semantic meanings of the tokens.

Example:

input: "I love AI" Tokenized: [‘I’, ‘love’, ‘AI’] Embedding: Converts each token into a vector, e.g., [0.1, 0.3, ...], [0.5, 0.2, ...], [0.7, 0.9, ...].

Positional Encoding:

Since the Transformer does not use recurrent or convolutional layers, it has no inherent understanding of the token positions. Therefore, positional encodings are added to the input embeddings to inject information about the position of tokens in the sequence. The positional encoding is a fixed function of the position (e.g., sine and cosine functions).



GEN-AI Handout - 1

Encoder-Decoder Structure:

The Transformer model consists of two main components:

Encoder:

Processes the input sequence and generates contextualized embeddings.

Decoder:

Generates the output sequence, typically used for tasks like language translation. Each of these components has multiple layers, and each layer consists of two sub-layers (for the encoder) and three sub-layers (for the decoder).

Encoder Block:

The encoder processes the input in multiple layers, each with two key sub-layers:

Multi-Head Self-Attention:

This allows the model to attend to different positions in the sequence, learning the relationships between tokens. The attention mechanism is described below.

Feed Forward Neural Network:

A fully connected neural network is applied to the output of the attention mechanism. It is applied independently to each position in the sequence. After each sub-layer, residual connections (skip connections) are used, followed by layer normalization.

Attention Mechanism:

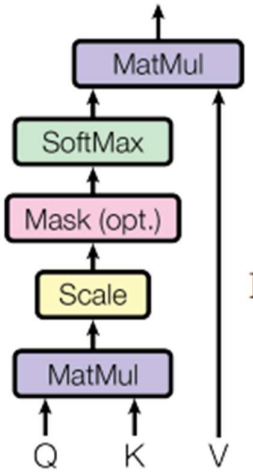
Self-Attention allows the model to weigh the importance of different tokens in the input sequence relative to each other. The input tokens are first converted into embeddings and combined with positional encodings. The embeddings are linearly projected into three vectors: Query (Q), Key (K), and Value (V).The scaled dot-product attention ensures that each token attends to all other tokens in the sequence to capture context.

Multi-Head Attention:

The Transformer splits the attention mechanism into multiple heads, allowing the model to focus on different parts of the sequence independently. After applying attention, the outputs of all heads are concatenated and projected back into the model's hidden space.

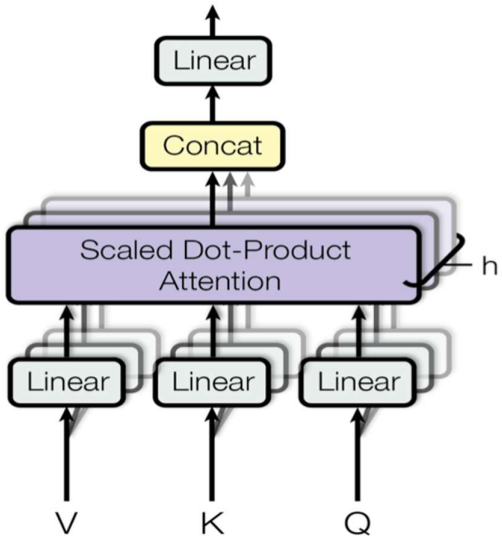
MultiHead(Q,K,V)=Concat(head1,head2,...,headh)W

GEN-AI Handout - 1



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Decoder Block:

Each decoder layer has three primary sub-layers:

Masked Multi-Head Self-Attention:

Like the encoder's self-attention but includes a mask to prevent the decoder from attending to future tokens during training. This ensures that the prediction for a token only depends on the known outputs up to that point.

Multi-Head Encoder-Decoder Attention:

This layer allows the decoder to attend to the encoder's output. It helps the model focus on relevant parts of the input sequence when generating each token in the output. Queries: Derived from the previous decoder layer. Keys and Values: Derived from the encoder's output. Attention Calculation: Like the self-attention mechanism but across encoder-decoder interactions.

GEN-AI Handout - 1

Feed Forward Neural Network:

After the attention mechanism, each token is passed through a feed-forward neural network independently to extract more features. Applied to the output of the attention mechanism. Identical to the one used in the encoder, providing additional non-linear transformations.

Final Linear Layer and SoftMax:

The output from the last decoder layer is passed through a fully connected linear layer that projects it back into the vocabulary space. A SoftMax function is applied to produce probabilities over the possible output tokens. The token with the highest probability is selected as the predicted next word.

Attention Weights:

The self-attention mechanism computes a weight matrix that represents how much each token should focus on other tokens. This allows the model to capture long-range dependencies and context across the sequence.

Training the Transformer:

The Transformer is trained using a loss function like cross-entropy loss, which compares the predicted sequence to the ground truth. Backpropagation is used to adjust the model's weights based on the gradient of the loss function.

Applications of Transformer:

NLP:

Translation, text generation, question answering, sentiment analysis.

Computer Vision:

Image classification, object detection, image generation.

Speech Processing:

Speech recognition, text-to-speech synthesis.

Reinforcement Learning:

Gaming, robotics.

Time-Series:

Financial forecasting, weather prediction.

Healthcare:

Protein folding, genomics, medical record analysis.

Autonomous Systems:

Self-driving cars, perception, trajectory prediction.

Recommendation Systems:

Content recommendations, personalized search.

Code Generation:

Code completion, bug detection.